

```
int schd_init(void)
{
    int ret;
    struct device *dev_ret;
    if ((ret = alloc_chrdev_region(&dev, FIRST_MINOR, MINOR_CNT,
    "mqd")) < 0)
    {
        return -1;
    }
    printk("Major Nr: %d\n", MAJOR(dev));
    cdev_init(&c_dev, &fops);
    if ((ret = cdev_add(&c_dev, dev, MINOR_CNT)) < 0)
    {
        unregister_chrdev_region(dev, MINOR_CNT);
        return ret;
    }
    if (IS_ERR(cl = class_create(THIS_MODULE, "chardrv")))
    {
        cdev_del(&c_dev);
        unregister_chrdev_region(dev, MINOR_CNT);
        return PTR_ERR(cl);
    }
    if (IS_ERR(dev_ret = device_create(cl, NULL, dev, NULL,
    "mychar%d", 0)))
    {
        class_destroy(cl);
        cdev_del(&c_dev);
        unregister_chrdev_region(dev, MINOR_CNT);
        return PTR_ERR(dev_ret);
    }
    return 0;
}

void schd_cleanup(void)
{
    printk(KERN_INFO " Inside cleanup_module\n");
    device_destroy(cl, dev);
    class_destroy(cl);
    cdev_del(&c_dev);
    unregister_chrdev_region(dev, MINOR_CNT);
}

module_init(schd_init);
module_exit(schd_cleanup);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Pradeep Tewari");
MODULE_DESCRIPTION("Demo for Waiting on an Event");
```

The above example is the same as that of previous articles, except that the *flag* variable is used to signal the event. *schd_init()* and *schd_exit()* are initialisation and exit functions for the driver, respectively. Any process invoking the *read* on the device file would be blocked. Let's have a demo on the same, assuming that the module is compiled as *wait.ko*:

```
$ insmod wait.ko
Major Nr: 250
$ cat /dev/mychar0
Inside open
Inside read
Scheduling out
```

As seen above, upon loading the driver, we get the device file with the name *mychar0* in the */dev* directory. Executing the *cat* command will invoke the *read* function in the driver. So, this will get the process blocked. Let's open another shell and try to wake up the process by invoking the *write* on the device file.

```
$ echo 1 > /dev/mychar0
Inside open
Inside write
Interrupted by signal
Scheduling out
Inside close
```

As seen above, executing the *echo* command on the device file will invoke the *write* function in the driver, which in turn will wake up the process. So, does this get our process unblocked? Not really. The process is still blocked as the condition for waking up is not satisfied. *echo* will set the *flag* to '1' and this doesn't meet our wake up condition of *flag* being set to 'y'. So, how do we wake the process up? No points for guessing the right answer — just set the flag to 'y'. Let's have a look at the following sample run:

```
$ echo y > /dev/mychar0
Inside open
Inside write
Woken up
Inside close
Inside close
```

This will wake up the process as the condition of *flag* being set to value 'y' is satisfied.

So this seems to be more realistic as our process is waiting on some event, rather than unconditional waiting. But this is still manual waiting, wherein everything is being managed by a programmer and, as such, is prone to several synchronisation bugs. So, does the kernel provide any programming constructs to make the programmer's life simpler? Of course it does. To explore more on the wait mechanism in the Linux kernel, stay tuned to the next article on this subject. 

By: Pradeep Tewari

The author works at Intel, Bengaluru. He shares his learning on Linux kernel and embedded systems through his weekend workshops. Learn more about his experiments at <https://sysplay.in>. He can be reached at pradeep_zenith@hotmail.com.